From Games to Graphs: Categorical Composition of Genetic Algorithms Across Domains

Lyra (lyra-claude) Claudius (11o1111o11oo1o1o) Robin Langer
Nick Meinhold

Abstract

Genetic algorithms share a common compositional structure: evaluate, select, recombine, mutate. This structure is typically hidden inside imperative for-loops, making it difficult to reason about how strategies compose and what properties their compositions preserve. We formalize GA operators as morphisms in a Kleisli category, where the monad carries randomness, configuration, and logging. In this framework, a full evolutionary run is a single composite arrow, and strategies — generational, hourglass, island, adaptive — are higher-order operations on these arrows: functors on a category of pipelines.

We demonstrate the framework across three domains: checkers evaluation (real-valued weight vectors), maze topology (binary genomes), and symbolic regression (expression trees). The categorical structure is invariant across domains — only the objects (genome types) and specific morphisms (fitness, mutation) change. We present three results:

1. **Diversity fingerprints.** Different strategy compositions produce qualitatively distinct diversity trajectories — monotonic decline (flat), spike-crash-rebound (hourglass), steady maintenance (island), spike-then-collapse (adaptive). These signatures are stable across genome types and fitness landscapes.
2. **The Strict/Lax Dichotomy Theorem.** The island strategy functor preserves sequential composition strictly if and only if no migration occurs. Any nonzero migration — even one individual per thousand generations — produces a uniformly lax functor. The laxity magnitude is independent of migration frequency and composition boundary position.
3. **Three-level categorical composition.** Operators compose into pipelines, pipelines compose into strategies, strategies compose into multi-population systems. Each level has the same categorical structure, enabling recursive reasoning about evolutionary systems.

The framework is implemented in Haskell (75 tests, 12 modules, ∼5500 lines), translated from a working Rust GA system. We position the work against the INSTINCT evolutionary computation literature corpus, where knowledge graph analysis reveals that game-theoretic coevolution and structural optimization are disconnected clusters. The categorical framework provides the shared formal language they lack.

1 Introduction: Samuel's Insight, Revisited

In 1959, Arthur Samuel wrote a program that learned to play checkers by adjusting the weights of a linear evaluation function through self-play. The program did not know what checkers was. It knew only that certain weight configurations led to more wins than others, and it adjusted weights in the direction of success. Samuel called this "machine learning." Today we would also call it evolutionary optimization: the weights are a genome, the win rate is fitness, and the adjustment process is a mutation-selection loop.

Sixty years later, the same algorithmic shape appears in a different domain. CianLR's `mazegen-rs` evolves maze topologies: the genome is a set of edges in a grid graph, fitness measures difficulty and aesthetic properties, and mutation swaps edges while preserving the spanning tree invariant. The mazes look nothing like checkers. The genomes are incompatible types. The fitness functions measure incommensurable quantities. And yet the algorithm — evaluate, select, recombine, mutate, repeat — is structurally identical.

This paper names that structure and shows what follows from naming it.

The structure is categorical. GA operators are morphisms (arrows) in a Kleisli category, where the monad carries the side effects that every genetic algorithm needs: randomness, configuration parameters,

and logging. Composition of morphisms is pipeline assembly. A full evolutionary run is a single composite arrow. Strategies — the high-level decisions about how to organize evolution (generational, steady-state, island, adaptive) — are functors: systematic transformations from one pipeline to another that preserve the compositional structure.

This is not a metaphor. When we say that the island model is a functor, we mean that it satisfies specific mathematical laws — and that when those laws are satisfied strictly versus laxly, the observable behavior of the system differs in predictable, measurable ways. The Strict/Lax Dichotomy Theorem (Section 6.3) is a concrete consequence: the island functor preserves sequential composition if and only if no migration occurs. One migrant changes everything.

The paper makes three contributions:

1. **A concrete categorical framework for GAs.** Not abstract nonsense — a working Haskell library where operators compose with `>>>:`, strategies compose with higher-order combinators, and the category laws hold by construction. We demonstrate it on three domains: checkers weight evolution, maze topology evolution, and symbolic regression via genetic programming.

2. **Empirical characterization of composition.** Different ways of composing the same operators produce qualitatively different population dynamics. We identify four distinct "diversity fingerprints" — flat, hourglass, island, adaptive — and show they are stable across genome types. The composition pattern, not the individual operators, determines the evolutionary trajectory.

3. **A bridge between disconnected literatures.** Knowledge graph analysis of the INSTINCT evolutionary computation corpus (76 papers, 555 concepts) reveals that game-theoretic coevolution and structural design optimization occupy disconnected clusters with zero concept overlap. Our categorical framework provides a formal language for stating what these communities share: the same morphism composition, different objects.

The framework was discovered, not designed. We began by translating a working Rust GA (`mazegen-rs`) into Haskell, and the categorical structure emerged from the type system. Section 4 documents this translation as a case study in how functional programming makes implicit structure explicit.

2 Background

2.1 A Minimal Category Theory for Evolutionary Biologists

You already think categorically. When you describe a genetic algorithm as "evaluate, then select, then recombine, then mutate," you are describing a composition of arrows in a category. When you observe that island migration works the same way regardless of whether the organisms are bitstrings or neural networks, you are observing a natural transformation. This section names the structure you are already using. Once named, it becomes something you can reason about formally — and the reasoning yields results (Section 6.3) that are not obvious from the algorithmic description alone.

What is a category?

A category $\mathbf{C}$ consists of:

1. A collection of **objects** (think: types of data).

2. For every pair of objects A, B, a collection of **arrows** (or **morphisms**) $f : A \to B$ (think: transformations from A-data to B-data).

3. For every object A, an **identity arrow** $\mathrm{id}_A : A \to A$ that does nothing.

4. A **composition operation**: given $f : A \to B$ and $g : B \to C$, their composite $g \circ f : A \to C$ applies f then g.

Composition must satisfy two laws:

- **Identity**: composing with the identity does nothing. $\mathrm{id} \circ f = f$ and $f \circ \mathrm{id} = f$.
- **Associativity**: parenthesization doesn't matter. $(h \circ g) \circ f = h \circ (g \circ f)$.

That is the entire definition. A category is a collection of things and transformations between them, where transformations compose associatively and there is a do-nothing transformation for each thing.

Example: Racing pigeons. Imagine you're trying to breed the fastest racing pigeon. You have a flock of 100 pigeons, and each generation you pick the fastest ones, breed them, and repeat. The "objects" are types of flocks: unscored flocks, scored flocks, selected flocks. The "arrows" are operations: evaluate (time each pigeon), select (keep the top 20), breed (produce offspring from pairs). Composing evaluate-then-select-then-breed gives you one generation step. That composite is itself an arrow — you can apply it again to get the next generation. **Composition closes over the same type.**

Example: Unix pipelines. The objects are stream formats (SAM, BED, VCF, plain text). The arrows are command-line tools: `samtools view` transforms SAM → BAM, `bedtools coverage` transforms BAM × BED → coverage table, `sort` transforms text → sorted text. The pipe operator `|` is composition. The identity is `cat`. A pipeline like

```
samtools view -F 4 aligned.bam | bedtools coverage -a genes.bed | sort -k5 -nr
```

is a composite arrow from SAM input to sorted coverage output. The pipeline is itself an arrow — you could pipe its output into another tool. Composition closes over the same type.

The GA lifecycle is a category

In a genetic algorithm, the objects are population types:

- `[genome]` — a list of unevaluated genomes (e.g., bitstrings, weight vectors, trees)
- `[scored genome]` — genomes paired with fitness values
- `[genome]` again, after selection and recombination strip the scores

The arrows are GA operators:

Operator	Arrow type	Pigeon version
Evaluate	`[genome]` → `[scored genome]`	Time each pigeon's flight
Select	`[scored genome]` → `[scored genome]`	Keep the fastest 20
Crossover	`[scored genome]` → `[scored genome]`	Breed pairs of survivors
Mutate	`[scored genome]` → `[scored genome]`	Random feather variation
Unwrap	`[scored genome]` → `[genome]`	Forget the timing data

One generation is the composition:

```
evaluate >>> select >>> crossover >>> mutate >>> unwrap
```

This composite is itself an arrow `[genome]` → `[genome]`. Running 100 generations is composing this arrow with itself 100 times. An entire evolutionary run is a single composite arrow.

The key property is **closure**: a pipeline of five operators is itself an operator. A strategy that runs one pipeline for 50 generations and then switches to another pipeline is itself a strategy. An island model containing four independent GAs with periodic migration is itself a GA. At every level of organization, the composite has the same type as its parts.

Where the monad enters

GA operators are not pure functions — they have **side effects**:

- **Randomness.** Selection, crossover, and mutation all need random numbers.
- **Configuration.** Tournament size, mutation rate, population size — read by every operator but never modified.
- **Logging.** Each generation produces statistics that accumulate into a history.

A **monad** solves this by packaging the side effects into a context that the composition operator handles automatically. Instead of pure arrows $A \to B$, we work with **effectful arrows** $A \to M(B)$, where M carries all three effects.

These effectful arrows form their own category — the **Kleisli category** for M. Composition in the Kleisli category applies the first arrow, extracts the result from inside M, and feeds it to the second arrow, while M's internal machinery threads the random seed, propagates the configuration, and accumulates the log. The individual operators never see this plumbing.

The GA pipeline `evaluate` $\ggg$ `select` $\ggg$ `crossover` $\ggg$ `mutate` $\ggg$ `unwrap` is a composition in the Kleisli category. Each $\ggg$ threads the effects invisibly. The result is a single effectful arrow that, when executed with a configuration and a random seed, produces a new population and a log of what happened.

Functors and natural transformations

A **functor** $F : \mathbf{C} \to \mathbf{D}$ maps between categories, sending objects to objects and arrows to arrows, preserving composition and identities. In the island model (Section 3), the island functor takes a single-population GA and produces a multi-population GA.

A **natural transformation** $\eta : F \to G$ between two functors is a family of arrows $\eta_A : F(A) \to G(A)$ satisfying the naturality condition: the diagram commutes regardless of which arrow f you apply.

Migration is a natural transformation. It moves individuals between islands without inspecting or modifying them — a structural operation on populations, not a content operation on individuals. In pigeon terms: transferring birds between lofts doesn't change the birds. The naturality of migration is what enables the Strict/Lax Dichotomy Theorem (Section 6.3).

Summary of categorical vocabulary

Category theory	In this paper	Pigeon intuition
Object	Population type	What kind of data each pigeon carries
Arrow (morphism)	GA operator	A step in the breeding program
Composition	Pipeline assembly ($\ggg$)	"Do this, then do that"
Identity	Pass-through operator	A generation where nothing happens
Monad	Effect context (M)	The bookkeeping: dice rolls, breeding rules, record-keeping
Kleisli arrow	Effectful GA operator	An operator that rolls dice, checks rules, writes records
Kleisli composition	Effectful pipeline ($\ggg$)	Compose steps while threading all bookkeeping automatically
Functor	Island model	Lift one loft to five lofts
Natural transformation	Migration	Move pigeons between lofts without inspecting them

2.2 co-Kleisli Composition and Format Strings

Our framework is inspired by Cale Gibbard's `category-printf` library, which uses the *dual* categorical construction to build type-safe format strings.

Where we work in the Kleisli category (arrows $A \to M(B)$, where M is a monad accumulating effects), `category-printf` works in the co-Kleisli category (arrows $W(A) \to B$, where W is a comonad providing context). Specifically, $W = ((\to)m)$ for a monoid m, so co-Kleisli arrows have type $(m \to A) \to B$.

Composition accumulates what the pipeline **demands** (argument types for the format string), just as Kleisli composition accumulates what the pipeline **does** (effects of the GA operators). This duality is not coincidental — it reflects the fundamental asymmetry between producers and consumers in categorical composition:

	co-Kleisli (category-printf)	**Kleisli (categorical-evolution)**
Direction	Comonad provides context	Monad carries effects
Accumulates	Argument types (what you demand)	Side effects (what you do)
Example	`"Hello " . s . "!"` builds a format demanding `String`	`evaluate` ⋙ `select` ⋙ `mutate` builds a pipeline that rolls dice, reads config, writes logs

The connection is pedagogical rather than technical: it shows that the Kleisli/co-Kleisli pattern of "composition accumulates structure" appears across very different domains.

2.3 The Evolution Monad

Our concrete monad stack is:

```
type EvoM = ReaderT GAConfig (WriterT GALog (State StdGen))
```

Three layers, each contributing one effect:

- **ReaderT GAConfig**: Tournament size, mutation rate, crossover rate, population size. Read-only — every operator can access configuration without passing it as an argument.
- **WriterT GALog**: Generation-by-generation fitness statistics, diversity measurements. Write-only — operators append log entries that accumulate over the run.
- **State StdGen**: The random number generator state. Threaded through every stochastic operation (selection, crossover, mutation).

A GA operator in this monad has type:

```
newtype GeneticOp m a b = GeneticOp { runOp :: [a] -> m [b] }
```

The `[a] → m [b]` signature says: take a population of `a`-individuals, produce a population of `b`-individuals, possibly using randomness, configuration, and logging along the way.

Operators compose via Kleisli composition:

```
(>>>:) :: Monad m => GeneticOp m a b -> GeneticOp m b c -> GeneticOp m a c
(GeneticOp f) >>>: (GeneticOp g) = GeneticOp (\pop -> f pop >>= g)
```

This is the standard monadic bind, lifted to operate on populations. The monad laws guarantee associativity and identity, so `GeneticOp m` forms a category.

3 Two Domains, One Category

We apply the framework to two concrete domains chosen for maximum dissimilarity: checkers (game strategy, real-valued genomes, adversarial fitness) and mazes (structural design, binary genomes, intrinsic fitness). The point is not that either domain is novel, but that the categorical pipeline is *identical* across both.

3.1 Checkers: Evolving Evaluation Functions

Arthur Samuel's 1959 checkers program evaluated board positions using a weighted sum of features: material advantage, king count, center control, advancement, mobility, and several others. The weights were learned through self-play — a process that is, structurally, a genetic algorithm with population size 1.

Feature	What it measures
Material	Piece count advantage
Kings	King count advantage
Back row	Pieces on the back row (defensive)
Center	Pieces controlling center squares
Advancement	Average distance toward promotion
Mobility	Number of legal moves available
Vulnerable	Pieces that can be captured
Protected	Pieces defended by allies
King center	Kings controlling center
Edge	Pieces on edge squares

We evolve a population of 30 weight vectors, each encoding 10 Samuel-style features:

Each weight is a real number in $[-5, 5]$. Fitness is win rate against two opponent strategies (2 games each, alternating colors). Mutation is Gaussian perturbation; crossover is one-point on the weight vector.

In the categorical framework:

```
evaluate (winRate opponents)
  >>> tournamentSelect
  >>> onePointCrossover
  >>> pointMutate gaussianPerturb
```

This pipeline has type `GeneticOp EvoM [Double] [Double]` — a Kleisli arrow from populations of weight vectors to populations of weight vectors.

The evolved weights beat hand-tuned defaults 20–0. The GA discovers that vulnerability assessment (pieces under threat) carries the highest weight — an aggressive strategy that prioritizes threat awareness over simple material counting.

3.2 Mazes: Evolving Topological Structures

The maze domain, adapted from `mazegen-rs`, evolves 6×6 grid mazes. The genome is a 60-bit binary vector encoding wall presence. Fitness is a composite of three BFS-derived metrics: solution path length (longer is harder), dead-end ratio (more dead ends increase difficulty), and branching factor (more choices create more interesting navigation).

In the categorical framework:

```
evaluate (mazeFitness 6 6)
  >>> tournamentSelect
  >>> onePointCrossover
  >>> pointMutate bitFlip
```

This pipeline has type `GeneticOp EvoM [Bool] [Bool]` — a Kleisli arrow from populations of binary genomes to populations of binary genomes.

The genomes are incompatible types (`[Double]` vs `[Bool]`). The fitness functions measure incommensurable quantities (game outcomes vs maze properties). The mutation operators are domain-specific (Gaussian perturbation vs bit flip). And yet the pipeline has the *same shape*. The composition operator $\ggg$ is identical. The monad stack is identical. The strategy combinators — hourglass, island, adaptive — work unchanged on both domains.

3.3 What Changes, What Stays

This is exactly what category theory predicts: the structure lives in the arrows and their composition, not in the objects. Changing the objects (genome types) is a change of *content*; the categorical *structure* is preserved.

Aspect	Checkers	Mazes	Invariant?
Genome type	`[Double]`	`[Bool]`	Changes
Fitness function	Win rate	BFS composite	Changes
Mutation	Gaussian perturb	Bit flip	Changes
Pipeline shape	evaluate ⋙ select ⋙ cross ⋙ mutate	Same	**Invariant**
Monad stack	`ReaderT GAConfig (WriterT GALog (State StdGen))`	Same	**Invariant**
Strategy combinators	hourglass, island, adaptive	Same	**Invariant**
Composition operator	`>>>:`	Same	**Invariant**
Category laws	Associativity, identity	Same	**Invariant**

We add a third domain — symbolic regression via genetic programming (expression trees, tree crossover, parsimony-pressured fitness) — in the experiments section. The categorical pipeline is again identical. Three domains, three incompatible genome types, one compositional structure.

4 From Rust to Haskell: Revealing Hidden Composition

The categorical framework did not emerge from abstract theory imposed on a toy problem. It was discovered by translating a working imperative genetic algorithm — `mazegen-rs`, a Rust maze evolution system — into Haskell. The translation changed nothing about the algorithm's behavior. It changed what is *visible* about the algorithm's structure.

4.1 The Rust Implementation

The Rust GA in `mazegen-rs` follows a standard imperative pattern. The evolution loop (approximately 75 lines) manually sequences operations:

```
for gen in 1..=config.generations {
    let mut next_genomes: Vec<Genome> = Vec::new();

    // Elitism
    for i in 0..elite {
        next_genomes.push(genomes[i].clone());
    }

    // Fill rest with offspring
    while next_genomes.len() < config.pop_size {
        let a = tournament_select(&fitnesses,
                 config.tournament_size, &mut rng);
        let b = tournament_select(&fitnesses,
                 config.tournament_size, &mut rng);
        let mut child = genomes[a].crossover(
                        &genomes[b], &mut rng);

        if rng.gen::<f64>() < config.mutation_rate {
            child.mutate(&mut rng);
        }

        let f = evaluate(&child, &config.target);
        next_fitnesses.push(f);
        next_genomes.push(child);
    }

    // Sort by fitness ...
}
```

Three things are hidden in this code:

1. **Selection returns an index**, not an individual. The connection between selection and crossover is implicit, mediated by an integer.
2. **Composition is manual**. The sequence select → crossover → mutate → evaluate exists only as ordered statements in a loop body.
3. **Effects are threaded explicitly**. Randomness (`&mut rng`), configuration (`config.tournament_size`), and evaluation target are passed as separate arguments to every function.

4.2 The Haskell Translation

The same algorithm in the categorical framework:

```
generationStep fitFunc mutFunc gen =
  evaluate fitFunc
    >>>: logGeneration gen
    >>>: elitistSelect
    >>>: onePointCrossover
    >>>: pointMutate mutFunc
```

Six lines. Each line is a morphism; `>>>:` is Kleisli composition. The type signature states exactly what happens: a function from populations to populations, carried in the evolution monad.

4.3 What the Translation Reveals

Selection becomes a morphism. In Rust, `tournament_select` returns an index; the caller looks up `genomes[a]`. In Haskell, `tournamentSelect :: GeneticOp EvoM (Scored a) (Scored a)` — a morphism from scored populations to scored populations. No indices, no parallel arrays.

The pipeline becomes a first-class value. In Rust, the generation step is a block of code inside a `for` loop. In Haskell, it is a value of type `GeneticOp EvoM [a] [a]` that can be composed with other strategies, lifted into an island functor, or analyzed by the landscape module.

Effects unify through the monad. Rust threads randomness, configuration, and logging separately. Haskell's `EvoM` monad unifies all three — adding a new effect changes the monad stack once, not every function signature.

4.4 The Same Algorithm, Different Visibility

Aspect	Rust	Haskell
Pipeline	Implicit (loop body)	Explicit (`GeneticOp m a b`)
Selection	Returns index	Returns individual
Composition	Manual sequencing	Type-checked `>>>:`
Effects	Threaded explicitly	Unified in `EvoM`
Reuse	Copy-paste the loop	Compose morphisms

The Haskell version does not add any new capability. It makes existing capability *composable*. And composability is what enables the three-level tower (operators → pipelines → strategies), the island strategy functor, and the dichotomy theorem.

We do not claim that GA practitioners should switch to Haskell. We claim that the categorical structure is present in *every* GA implementation — the types are there whether or not the language can express them.

5 Connecting the Islands: Literature Positioning

5.1 The Knowledge Graph

Analysis of the INSTINCT evolutionary computation corpus — 76 papers, 555 concept nodes, 446 edges — reveals a fragmented landscape. The papers cluster into five communities: neuroevolution, morphology-control co-evolution, open-ended evolution (OEE), quality-diversity (QD), and game-theoretic coevolution.

5.2 The Game-Theory Island

Game-theoretic coevolution has zero concept overlap with three of the four other clusters. The only bridge concept is "fitness," shared with morphology-control — a generic term that provides no structural connection. This is the deepest structural hole in the corpus.

This isolation is surprising. Game-theoretic coevolution (evolving strategies for competitive games) and morphological evolution (evolving robot body plans) both use the same algorithmic machinery: population-based search with selection, crossover, and mutation. But the communities use different vocabularies, cite different foundational papers, and attend different conferences. The structure is shared; the language is not.

5.3 Bridge Papers and Our Contribution

Three papers appear most frequently as bridges across structural holes:

- **Stanley & Miikkulainen (2002)** NEAT: bridges 6 of 10 structural holes, primarily through neuroevolution's connections to other fields.
- **Cully et al. (2014)** Robots That Can Adapt: bridges morphology-control to quality-diversity.
- **Lehman & Stanley (2011)** Novelty Search: bridges neuroevolution to OEE and OEE to QD.

Our contribution bridges **game-theory to structural-design** — the deepest hole — by providing a formal language (categorical composition) that states precisely what the domains share. The bridge is not another algorithm or application. It is a *vocabulary*: the same morphism composition, parameterized by different objects.

5.4 The Neuroevolution-OEE Gap

The bridge score between neuroevolution and open-ended evolution is 22, but this is inflated by tool-usage citations — NEAT used as a component in OEE systems, not studied as a source of open-ended complexity. Nobody has coupled growing-topology neuroevolution with growing-complexity environments. The categorical framework offers a language for describing this coupling: functors between behavior categories of different dimensions, where the functor itself evolves.

This remains a research direction, not a result. We mention it because the knowledge graph analysis identifies it clearly, and the categorical vocabulary provides the right level of abstraction for stating the problem.

6 Experiments

6.1 Four Strategy Compositions on Symbolic Regression

We compare four compositions of evolutionary strategies on the problem of discovering $y = x^2 + x$ from 21 training points:

1. **Flat Generational** — 50 generations of standard generational GA. A single morphism applied 50 times.
2. **Hourglass** — `sequential(explore 15, converge 10, diversify 25)`. Three phases inspired by the developmental hourglass model.

3. **Island GA** — 4 subpopulations with ring migration every 5 generations. A functor from the strategy category to itself.

4. **Adaptive** — `explore` with plateau detection switching to `focus`. A conditional composition with a runtime predicate.

Population: 60 expression trees per strategy.
All four strategies find the exact expression $(x \cdot x) + x$:

Strategy	Best Fitness	Final GenoDiv	Final PhenoDiv
Flat	-0.05	5.71	444.01
Hourglass	-0.05	7.07	277.93
Island	-0.05	5.83	615.42
Adaptive	-0.05	1.92	213.42

The key result is not fitness (all reach the optimum) but **diversity trajectories**:

Table 1: Genotypic diversity over time (tree size variance)

Gen	Flat	Hourglass	Island	Adaptive
0	22.98	22.98	22.98	22.98
5	27.71	**58.64**	18.04	**52.88**
10	14.80	**44.88**	11.76	**41.81**
15	11.21	15.82	6.67	4.22
20	6.33	**3.37**	6.24	5.43
25	6.57	4.00	7.06	3.33
30	7.19	6.86	5.82	4.40
50	5.71	7.07	5.83	1.92

Four distinct fingerprints:

- **Flat**: Monotonic decline 22.98 → 5.71. No structural intervention prevents convergence.
- **Hourglass**: Spike to 58.64, crash to 3.37, rebound to 7.07. Three phases clearly visible.
- **Island**: Stable around 6–7 after initial drop. Migration maintains diversity without dramatic oscillations.
- **Adaptive**: Spike to 52.88, then collapse to 1.92. Plateau detection triggers early convergence with no recovery.

These signatures demonstrate that the **composition pattern** — not the individual operators — determines population dynamics.

6.2 Cross-Domain Robustness

We apply flat, hourglass, and island to checkers (10 real-valued weights, pop 30) and mazes (60-bit binary, pop 30):

The qualitative pattern — flat < island < hourglass for final genotypic diversity — holds across all three genome types and fitness landscapes. Checkers saturates quickly (all reach 1.0 win rate), compressing diversity differences. Mazes, with a richer landscape, show the clearest separation.

Table 2: Cross-domain genotypic diversity

Gen	C:Flat	C:Hrgl	C:Isld	M:Flat	M:Hrgl	M:Isld
0	0.49	0.49	0.49	0.47	0.47	0.47
5	0.34	0.32	0.23	0.21	0.51	0.37
10	0.33	0.33	0.47	0.26	0.29	0.31
15	0.31	0.32	0.44	0.26	0.46	0.28
20	0.30	0.30	0.34	0.27	0.46	0.32

Table 3: Final comparison

Domain	Strategy	Best Fit	GenoDiv	PhenoDiv
Checkers	Flat	1.0	0.30	0.0
Checkers	Hourglass	1.0	0.30	0.0
Checkers	Island	1.0	0.34	0.0
Maze	Flat	0.70	0.27	40.36
Maze	Hourglass	0.68	0.46	19.96
Maze	Island	0.73	0.32	40.14

6.3 The Strict/Lax Dichotomy Theorem

We test whether the island strategy functor preserves sequential composition: does $I(S_1; S_2) = I(S_1); I(S_2)$?

Migration frequency sweep (4-island ring topology, 20-bit OneMax):

Binary transition: at freq=40 (no migration in the window), the functor is **strict**. At every other frequency, it is **uniformly lax** with divergence $\sim 0.75 \pm 0.05$, independent of frequency.

Boundary position sweep (freq=5, boundary from gen 15 to 25):

Divergence is **uniform** ($\sim 0.75 \pm 0.05$) regardless of whether the boundary coincides with a migration event. This falsifies the naive prediction that divergence should be zero at non-coincidence boundaries.

Theorem 6.1 (Strict/Lax Dichotomy)**.** *Let $I(\mu, \text{freq}, n, \text{topo})$ be the island strategy functor. For any strategies S_1, S_2:*

1. *If $\mu = 0$ or $\text{freq} > |S_1| + |S_2|$ (no migration events), then $I(S_1; S_2) = I(S_1); I(S_2)$ — the functor is **strict**.*
2. *Otherwise, the functor is **lax**: $I(S_1; S_2) \cong I(S_1); I(S_2)$ with laxator magnitude D^* that is independent of boundary position and number of affected migration events, determined asymptotically by the spectral gap of the migration Markov chain.*

The laxator represents chaotic amplification of the perturbation introduced by resetting the migration schedule at the composition boundary. Even a single displaced migration event produces the same asymptotic divergence as displacing many.

In pigeon terms: if you split a 40-generation island breeding program into two 20-generation phases, the results differ from running it straight through — *unless* no birds ever migrated between lofts during the full program. One transferred pigeon, at any point, changes everything. The schedule disruption cascades.

6.4 Evolved Artifacts

Checkers evaluation weights (hourglass, 20 generations):

```
material:   1.53   vulnerable: 2.14
kings:      0.31   protected: -1.96
back_row:  -0.96   king_center: -0.45
center:     0.36   edge:       1.12
advancement: -1.41
```

Table 4: Island functor laxity vs. migration frequency

Freq	Sched Diff	Pop Divergence	Hamming Div	Fitness Diff
2	1	0.744	0.110	0.0
5	1	0.812	0.102	0.0
10	1	0.756	0.096	0.0
20	1	0.819	0.129	0.0
40	**0**	**0.000**	**0.000**	**0.0**

Table 5: Divergence vs. composition boundary position

Boundary	Hits Migration?	Pop Divergence	Hamming Div
15	YES	0.794	0.105
17	no	0.725	0.098
20	YES	0.812	0.102
23	no	0.744	0.108
25	YES	0.800	0.102

```
mobility:  -1.05
```

The evolved strategy prioritizes vulnerability assessment above all else — an aggressive posture that beats defaults 20–0.

GP expression (all strategies): $(x \cdot x) + x$ — exact recovery of the target in minimal 5-node form.

Best evolved maze (island, 20 gen):

```
+---+---+---+---+---+---+
| S |   |               |
+   +---+   +   +---+---+
|       |   |       |   |
+   +   +   +   +   +   +
|           |   |       |
+   +   +   +---+---+   +
|       |   |   |       |
+---+   +---+   +   +   +
|       |   |   |       |
+---+---+---+   +---+   +
|       |             G |
+---+---+---+---+---+---+
```

7 Discussion

7.1 What the Category Reveals

The categorical framework is not just a different notation for standard GAs. It enables three things that imperative implementations cannot:

Formal composition reasoning. The Strict/Lax Dichotomy Theorem is a statement about functorial composition that has no natural expression in imperative code. You cannot ask "does this for-loop preserve sequential composition?" because the for-loop does not have a type that admits the question. The categorical framework gives evolutionary strategies a type — a morphism in a Kleisli category — and composition questions become type-level questions.

Domain-independent strategy design. The hourglass strategy was designed once and applied to three domains without modification. This is possible because the strategy operates on the *categorical structure*

(morphism composition) rather than on the *content* (genome representation). In imperative code, reusing a strategy across domains requires either copy-paste or a framework with sufficient abstraction — and existing GA frameworks (DEAP, ECJ, jMetal) typically abstract over representation but not over compositional strategy.

Diversity fingerprints as invariants. The observation that flat, hourglass, island, and adaptive produce qualitatively distinct diversity trajectories — and that these trajectories are stable across domains — is a statement about the compositional structure. The fingerprint is determined by how the operators are composed, not by what they operate on. This suggests a general principle: the dynamics of a categorical system can be read from its composition graph.

7.2 Limitations

Scale. Our experiments use small populations (30–60) and short runs (20–50 generations) on deliberately simple problems. The diversity fingerprints may not survive scaling to industrial-size problems with thousands of generations. The Dichotomy Theorem's uniform laxity result was demonstrated on 20-bit OneMax; whether D^* remains uniform on rugged, high-dimensional landscapes is an open question.

Statistical power. Results are from single runs per configuration (deterministic given a seed). Proper statistical analysis would require multiple seeds per condition — we traded breadth for depth, covering more conditions with less replication.

The Haskell constraint. While we argue that the categorical structure is language-independent, our implementation is in Haskell. Porting the framework to Python or C++ would require either a categorical library or disciplined use of interfaces/traits. The claim that the structure is universal is made but not demonstrated in a second language.

7.3 Future Directions

Coevolution as functor composition. Checkers naturally supports competitive coevolution; mazes could support cooperative coevolution (rooms that connect). Modeling both as functor compositions and comparing their categorical properties would extend the framework.

Variable-dimensional genotypes. NEAT's growing-topology networks and maze generators with variable grid sizes both involve genomes whose dimensionality changes over evolution. The categorical framework could handle this via functors between categories with different object types — but the implementation remains future work.

Landscape-aware strategy selection. The existing `recommendStrategy` function analyzes fitness landscape properties (ruggedness, neutrality, correlation length) and selects a strategy. Connecting landscape analysis to the diversity fingerprint theory — predicting which fingerprint a landscape demands — would close the loop between theory and practice.

8 Conclusion

A genetic algorithm is a composition of morphisms in a Kleisli category. This is not a metaphor — it is a precise mathematical statement with testable consequences. We have shown three such consequences: diversity fingerprints are determined by composition structure and stable across domains; the island functor exhibits a sharp strict/lax dichotomy at zero migration; and three levels of categorical composition (operators, pipelines, strategies) share the same algebraic structure.

The framework bridges two disconnected communities in evolutionary computation — game-theoretic coevolution and structural design optimization — by providing a formal language for their shared structure. The bridge is not a new algorithm. It is a vocabulary: the same objects and arrows, the same composition laws, the same functors. Different genomes, different fitness functions, same category.

The pigeon breeders knew this all along. Whether you're breeding for speed, endurance, or plumage, the breeding program has the same shape: evaluate, select, breed, repeat. Category theory just gives that shape a name.

Code Availability

The categorical-evolution framework is available at: https://github.com/lyra-claude/categorical-evolution
75 tests, 12 modules, ∼5500 lines of Haskell. Compiles with GHC 9.4.8.

References

[1] Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3), 210–229.

[2] Gibbard, C. (2023). category-printf: Category-theoretic approach to type-safe printf. Hackage.

[3] Shuck, S. (2023). mtl: Monad classes for transformers. Hackage.

[4] Stanley, K. O., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2), 99–127.

[5] Cully, A., Clune, J., Tarapore, D., & Mouret, J.-B. (2015). Robots that can adapt like animals. *Nature*, 521(7553), 503–507.

[6] Lehman, J., & Stanley, K. O. (2011). Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2), 189–223.

[7] Wright, S. (1931). Evolution in Mendelian populations. *Genetics*, 16(2), 97–159.

[8] Wang, R., et al. (2019). Paired open-ended trailblazer (POET). *arXiv:1901.01753*.

[9] Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1), 55–92.